Phoenix Trading Platform — Complete Architecture Guide

Version: 1.15.3 | **Date:** April 2026 | **Author:** Phoenix Engineering

Table of Contents

1. Project Overview
2. Core Philosophy — Agent-First Design
3. System Architecture
4. Agent Types
5. End-to-End Flow: Channel to Live Trades
6. Backtesting Pipeline (12-Step)
7. Live Trading Pipeline
8. Position Monitoring Sub-Agents
9. Inter-Agent Communication
10. Agent Gateway — The Orchestrator
11. Robinhood MCP Server
12. Risk Management Chain
13. ML Pipeline & Models
14. Discord Message Ingestion
15. Notification System
16. Scheduler & Cron Jobs
17. Agent Lifecycle State Machine
18. Database Schema
19. File System Layout
20. Dashboard (React Frontend)
21. API Endpoints Reference
22. Security & Sandboxing
23. Deployment Architecture
24. Connecting to Running Agents
25. Debugging & Troubleshooting
26. Technology Stack
27. Future Roadmap

1. Project Overview

Phoenix is a **multi-agent AI trading platform** built as a monorepo. Every trading agent is a **Claude Code session** — a real AI process that reads instructions, reasons about market signals, calls Python tools, and takes trades autonomously.

What makes Phoenix different: - Agents are NOT Python scripts or microservices. They are Claude Code AI sessions that can reason, debug, and self-correct. - The intelligence lives in the agents' instruction files (CLAUDE.md) and the tools they invoke. - Python code serves as focused, single-purpose tools that agents call — not the orchestration layer. - Every trade decision is auditable: full reasoning chain from signal to execution.

Tech Stack: Python 3.11+, Node.js 18+, FastAPI, React 18, PostgreSQL, Redis, Claude SDK, Robinhood API.

---

2. Core Philosophy — Agent-First Design

```
Traditional approach:       Phoenix approach:
Python code = the brain     Claude Agent = the brain
If/else logic decides       AI reasons about each signal
Fixed pipelines             Agent adapts, debugs, self-corrects
Manual monitoring           Sub-agents per position
```

Core principle: Claude SDK agents are the brain. Python tools are focused, single-purpose scripts the agents invoke. The Agent Gateway (`agent_gateway.py`) is the single entry point for all agent lifecycle operations.

---

3. System Architecture

High-Level Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          USER (You)                                  │
```

```
│    ┌──────────┐     ┌──────────┐     ┌──────────────┐          │
│    │ Dashboard │     │ WhatsApp │     │ Terminal     │          │
│    │ (React)   │     │ (replies) │     │ (SSH/docker) │          │
│    └──────────┘     └──────────┘     └──────────────┘          │
└────────┼──────────────────┼──────────────────┼──────────────────┘
         │                  │                  │
         │ REST + WS        │ Webhook          │ docker exec
         ▼                  ▼                  ▼
┌──────────────────────────────────────────────────────────────┐
│                      PHOENIX API (FastAPI)                    │
│   ┌──────────────────────────────────────────────────────┐   │
│   │ Routes: agents, trades, positions, agent-messages,   │   │
│   │         chat, morning-routine, whatsapp_webhook, ...  │   │
│   └──────────────────────────────────────────────────────┘   │
│   ┌──────────────────────────────────────────────────────┐   │
│   │ Services:                                            │   │
│   │  • AgentGateway       (spawns Claude Code sessions)  │   │
│   │  • NotificationDispatcher (DB + WS + WhatsApp)       │   │
│   │  • MessageIngestion   (Discord → DB + Redis)         │   │
│   │  • Scheduler          (cron jobs + heartbeat)        │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
      │                  │                  │
      ▼                  ▼                  ▼
┌──────────┐        ┌──────────┐     ┌──────────────────┐
│PostgreSQL│        │ Redis    │     │ Claude Code Sessions │
│  29+     │        │ • Streams │     │ (one per agent)  │
│ tables   │        │ • Pub/Sub │     │                  │
└──────────┘        │ • Cache  │     │ Each session is a │
                    └──────────┘     │ separate process │
                                     │ with its own     │
                                     │ working directory │
                                     └──────────────────┘
                                          │
                                ┌─────────┼─────────┐
                                ▼         ▼         ▼
                          ┌──────────┐ ┌──────────┐ ┌──────────┐
                          │ Backtester │ │ Live Analyst │ │ Sub-Agents │
                          │ (one-shot) │ │ (continuous) │ │ (per pos.) │
                          └──────────┘ └──────────┘ └──────────┘
                                │         │         │
                                ▼         ▼         ▼
                          ┌──────────────────────────────────┐
                          │          External Services       │
                          │ Discord • Robinhood • yfinance • Reddit │
                          │ Twitter • Unusual Whales • Anthropic API │
                          │ WhatsApp Cloud API • OpenAI      │
                          └──────────────────────────────────┘
```

The Three Layers

Layer 1: Control Plane (FastAPI + Postgres + Redis + Dashboard)

- **FastAPI app** — REST API + WebSocket gateway
- **PostgreSQL** — single source of truth for ALL state (29+ tables)
- **Redis** — pub/sub for inter-agent messages, streams for Discord, cache
- **React Dashboard** — Vite + Tailwind + Radix UI

Layer 2: Execution Plane (Claude Code Agents)

Each agent is a `claude-agent-sdk` Python `query()` invocation that spawns a Claude Code subprocess with: - A working directory (e.g., `/app/data/live_agents/{id}/`) - A `CLAUDE.md` instructions file - A `tools/` subfolder of Python scripts - `permission_mode="dontAsk"` for unattended operation - `allowed_tools=["Bash", "Read", "Write", "Edit", "Grep", "Glob"]`

Layer 3: Data Plane (External APIs + ML Models)

- **yfinance** — historical OHLCV with disk cache
- **Robinhood** — broker via `robin_stocks` + custom MCP server
- **Discord API** — channel monitoring
- **Unusual Whales API** — options flow data
- **Anthropic API** — Claude Code SDK + LLM analysis

---

4. Agent Types

Agent Type	Purpose	Lifecycle	Template Directory
Backtesting Agent	Pulls Discord history, trains ML models, discovers patterns	One-shot (~15-30 min)	`agents/backtesting/`
Analyst Agent	Live trading from Discord signals	Continuous (until stopped)	`agents/templates/live-trader-v1/`
Position Monitor	One per open position; finds optimal exit	Continuous (until position closes)	`agents/templates/position-monitor-agent/`
Unusual Whales Agent	Monitors options flow + dark pool prints	Continuous polling	`agents/templates/unusual-whales-agent/`

Agent Type	Purpose	Lifecycle	Template Directory
Social Sentiment Agent	Reddit + Twitter signal scanning	Continuous polling	`agents/templates/social-sentiment-agent/`
Strategy Agent	Rule-based strategies (EMA crossover, 52w levels)	Continuous polling	`agents/templates/strategy-agent/`
Supervisor Agent	Nightly AutoResearch — analyzes performance	Daily (one-shot)	`agents/templates/supervisor-agent/`

---

5. End-to-End Flow: Channel to Live Trades

Step 1 — User creates an agent in the dashboard

```
Dashboard → "+ New Agent" → 3-step wizard:
  1. Pick a connector (Discord channel)
  2. Set risk parameters (stop loss %, max position %, daily loss limit)
  3. Review and create
```

Step 2 — API spawns a backtesting Claude session

1. Inserts a new `Agent` row (status = `BACKTESTING`)
2. Inserts a new `AgentBacktest` row (status = `RUNNING`)
3. Calls `gateway.create_backtester(agent_id, backtest_id, config)`
4. Spawns a Claude Code session via `claude_agent_sdk.query()`

Step 3 — Claude Code runs the 12-step backtesting pipeline

(See Section 6 for full details)

Step 4 — Backtest completes → user reviews and approves

1. `_auto_create_analyst()` loads the manifest into the Agent record
2. Dashboard shows metrics (win rate, Sharpe, drawdown, patterns)
3. User clicks **Approve** → API stores Robinhood credentials → auto-spawns live agent

Step 5 — Live analyst agent runs continuously

1. Starts Discord listener → watches configured channel
2. For each signal → runs decision engine (parse → enrich → inference → risk → TA → execute)
3. After successful trade → spawns position monitor sub-agent

Step 6 — Position sub-agent monitors each trade

One Claude Code session per open position, running exit analysis every 30s-2min.

Step 7 — Inter-agent knowledge sharing

Agents share market intelligence, sell signals, and morning research via the agent message bus.

Step 8 — Notifications

Trade events fire WhatsApp notifications (entry, exit, morning briefing, agent wake-up).

Step 9 — Nightly AutoResearch

At 4:30 PM ET, the Supervisor agent collects the day's data, analyzes performance, proposes improvements, and mini-backtests each proposal.

---

6. Backtesting Pipeline (12-Step)

The backtesting agent reads `agents/backtesting/CLAUDE.md` and executes:

Step	Tool	Description
1	`transform.py`	Pull Discord history, parse trade signals
2	`enrich.py`	Add ~200 market features per trade
3	`embed_text.py`	Sentence-transformer vectors for Discord messages
4	`preprocess.py`	Train/val/test split
5	`model_selector.py`	Pick optimal models based on dataset size
6	`train_*.py`	Train selected models sequentially (memory-constrained)

Step	Tool	Description
7	`evaluate.py`	Evaluate all models, pick best
8	`explain.py`	Feature importance / explainability
9	`discover_patterns.py`	Multi-condition trading rule discovery
10	`llm_analysis.py`	LLM narrative interpretation of patterns
11	`validate_models.py`	Validate predictions on hold-out data
12	`create_live_agent.py`	Build manifest.json with rules, character, models

After each step, the agent calls `POST /api/v2/agents/{id}/backtest-progress` for real-time dashboard updates.

Output artifacts: - `transformed.parquet` — raw trade data from Discord - `enriched.parquet` — with ~200 market features - `models/` — trained ML model files - `model_selection.json` — which models were chosen and why - `patterns.json` — discovered trading patterns - `explainability.json` — top features - `live_agent/manifest.json` — configuration for the live agent

---

7. Live Trading Pipeline

```
Discord Channel
      │ (message posted)
      ▼
Message Ingestion Daemon
      │ (persists to channel_messages table + Redis stream)
      ▼
signal_listener.py (Redis consumer)
      │ (writes signal JSON file to agent workspace)
      ▼
Claude Agent reads signal file
      │
      ├── parse_signal.py → extract ticker, direction, price
      ├── enrich_single.py → 200+ market features
      ├── inference.py → ML model prediction (TRADE/SKIP)
      ├── Agent applies its own reasoning + learned rules
      ├── risk_check.py → position/exposure limits
      ├── technical_analysis.py → TA confirmation
      │
      ├── IF confidence < threshold → add to watchlist
```

```
      ├── IF paper mode → add to Robinhood watchlist
      └── IF live mode → execute_trade.py → Robinhood MCP order
                                  │
                                  ▼
                        Spawn position monitor sub-agent
```

Key Design: The Claude agent is the decision-maker, not a Python if/else chain. It can reason about edge cases, combine multiple signals, and explain its decisions in plain English.

---

8. Position Monitoring Sub-Agents

For each open position, a NEW Claude Code session spawns:

```
data/live_agents/{agent_id}/positions/{ticker}_{timestamp}/
├── CLAUDE.md           # Position monitor instructions
├── position.json       # Assigned position details
├── tools/
│   ├── exit_monitor.py     # Main monitoring loop
│   ├── exit_decision.py    # Combined exit analysis
│   ├── ta_check.py         # RSI, MACD, Bollinger, S/R
│   ├── mag7_correlation.py # MAG-7 stock correlation
│   └── discord_sell_signal.py # Analyst sell signal detection
└── exit_check_*.json   # Historical exit decisions
```

Monitoring cadence: - First 5 minutes: check every 30 seconds - After: check every 2 minutes (or 30s if urgency >= 50)

Exit decision combines: 1. Technical analysis (RSI, MACD, Bollinger Bands, support/resistance) 2. MAG-7 correlation (broad market direction) 3. Discord sell signal detection (if the analyst sells) 4. Risk levels (stop loss, take profit from backtesting data)

Actions: HOLD / PARTIAL_EXIT / FULL_EXIT based on combined urgency score.

---

9. Inter-Agent Communication

Persistent Layer (DB + REST)

```
Agent A → POST /api/v2/agent-messages → Postgres → Agent B polls → reads message
```

Real-time Layer (Redis Pub/Sub)

Messages also published to `phoenix:agent-knowledge:{agent_id}` for instant delivery.

Knowledge Intents (standardized message types):

- `MARKET_BRIEFING` — pre-market research summary
- `POSITION_UPDATE` — position P&L changes
- `RISK_ALERT` — risk threshold breached
- `EXIT_SIGNAL` — time to close a position
- `SELL_SIGNAL` — analyst is selling
- `UNUSUAL_FLOW` — options flow anomaly detected
- `MORNING_RESEARCH` — shared morning analysis

---

10. Agent Gateway — The Orchestrator

`apps/api/src/services/agent_gateway.py` is the **single entry point** for all agent operations:

Method	Purpose
`create_backtester()`	Spawn backtesting Claude session
`start_analyst_agent()`	Spawn live trading Claude session
`spawn_position_agent()`	Spawn position monitor sub-agent
`resume_agent()`	Resume a paused agent
`pause_agent()`	Pause a running agent
`chat_with_agent()`	Route chat messages through Claude SDK
`dispatch_trigger()`	Push typed triggers via Redis
`_auto_create_analyst()`	Auto-spawn analyst after backtest completes

Key features: - In-memory `_running_tasks` dict tracks all active asyncio tasks - `_session_ids` dict maps agent IDs to Claude Code session IDs for resume - Budget enforcement (Phase H7) before spawning any

agent - Credit error detection with clear user-facing messages - Heartbeat monitoring (30-minute stale timeout) - Auto-restart of stale analyst/live_trader agents

---

11. Robinhood MCP Server

`agents/templates/live-trader-v1/tools/robinhood_mcp.py` — a JSON-RPC over stdio MCP server that agents call for broker operations.

Available tools (25+): | Tool | Description | |------|----------| | `robinhood_login` | Authenticate with TOTP 2FA + session persistence | | `get_positions` | List stock positions | | `get_option_positions` | List options with Greeks, P&L | | `get_all_positions` | Combined stock + options view | | `get_account` / `get_account_snapshot` | Balance, buying power | | `get_quote` / `get_nbbo` | Real-time quotes | | `get_option_chain` / `get_option_greeks` | Options data | | `place_stock_order` / `place_option_order` | Execute trades | | `smart_limit_order` | Intelligent limit order placement | | `close_position` / `close_option_position` | Close positions | | `get_order_status` / `get_order_history` | Order tracking | | `get_watchlist` | Watchlist management |

Session persistence: Uses `robin_stocks` with pickle-based session storage (24-hour expiry) and TOTP auto-generation to avoid repeated 2FA.

---

12. Risk Management Chain

Every trade passes through 3 layers before execution:

```
Signal from Discord/UW/Reddit/Strategy
              │
              ▼
   ┌──────────────────┐
   │ 1. Agent-Level Risk │ ── Per-agent: position size, daily limits
   └──────────────────┘
              │ pass
              ▼
   ┌──────────────────┐
   │ 2. Execution Risk   │ ── Per-trade: stop loss valid, price reasonable
   └──────────────────┘
              │ pass
              ▼
   ┌──────────────────┐
```

```
    │ 3. Global Risk      │ ── Account-wide: total exposure, drawdown
    └─────────────────────┘
              │ pass
              ▼
        BROKER EXECUTION
        (Robinhood MCP)
```

---

13. ML Pipeline & Models

Training runs 8+ models sequentially (memory-constrained):

Model	Type	Best For
XGBoost	Gradient Boosting	General-purpose, fast
LightGBM	Gradient Boosting	Large datasets, categorical features
CatBoost	Gradient Boosting	Handles missing data natively
Random Forest	Ensemble	Baseline, interpretable
LSTM	Deep Learning	Sequential patterns
Transformer	Deep Learning	Attention-based temporal patterns
TFT (Temporal Fusion)	Deep Learning	Multi-horizon forecasting
TCN (Temporal Conv)	Deep Learning	Long-range dependencies
Hybrid Ensemble	Meta-learner	Combines best of all models

Features span ~200 attributes: - Price action (OHLCV, returns, gaps) - Technical indicators (RSI, MACD, Bollinger, ATR, etc.) - Volume analysis (VWAP, OBV, volume spikes) - Market context (S&P 500, VIX, sector rotation) - Time features (day of week, time of day, earnings proximity) - Sentiment (Discord message embeddings, FinBERT) - Options data (put/call ratio, unusual flow)

---

14. Discord Message Ingestion

```
Discord Channel → Discord API
                      │
                      ▼
             Message Ingestion Daemon
             (runs inside API lifespan)
                      │
            ┌─────────┴──────────┐
            ▼                    ▼
     channel_messages        Redis Stream
     (PostgreSQL)            stream:channel:{id}
            │                    │
            ▼                    ▼
     Dashboard Feed Tab    signal_listener.py
                           (agent tool)
```

Features: - Auto-starts on API boot for all active Discord connectors - Auto-refreshes every 5 minutes (restarts dead connectors, picks up new ones) - On-demand backfill via `POST /api/v2/agents/{id}/channel-messages/backfill` - Fan-out trigger: wakes agents via Redis trigger bus on new messages

---

15. Notification System

```
Trade event → NotificationDispatcher.dispatch()
                      │
            ┌─────────┴──────────┬──────────┐
            ▼         ▼         ▼         ▼
         ┌─────┐  ┌────────┐  ┌──────┐  ┌────────┐
         │ DB  │  │  WS    │  │WhatsApp│ │Twitter │
         │     │  │(Redis  │  │      │ │(future)│
         └─────┘  │Stream) │  └──────┘  └────────┘
                  └────────┘
```

Notification types: Agent wake-up, morning research briefing, trade entry, trade exit, position alerts.

---

16. Scheduler & Cron Jobs

Time (ET)	Job	Description
9:00 AM	Morning Briefing	Wake agents, pre-market research
4:30 PM	AutoResearch Supervisor	Analyze day's performance, propose improvements
4:45 PM	EOD Analysis	Enrich trade signals with outcomes
5:00 PM	Daily Summary	WhatsApp summary across all agents
Every 5 min	Heartbeat Check	Mark stale sessions, auto-restart agents, refresh ingestion

Heartbeat: Sessions without a heartbeat for 30+ minutes are marked stale. Analyst and live_trader agents are automatically restarted.

17. Agent Lifecycle State Machine

```
        ┌─────────┐
        │ CREATED │ ◄── User creates agent in dashboard
        └─────────┘
             │ Backtest spawned
             ▼
      ┌─────────────┐
      │ BACKTESTING │ ◄── Claude Code session running pipeline
      └─────────────┘
             │ Pipeline complete + manifest loaded
             ▼
   ┌──────────────────┐
   │ BACKTEST_COMPLETE │ ◄── User reviews metrics in dashboard
   └──────────────────┘
             │ POST /approve
             ▼
      ┌──────────┐          ┌─────────┐
      │ APPROVED │ or       │  PAPER  │
      └──────────┘          └─────────┘
             │ Auto-spawn live    │
             ▼                    ▼
        ┌─────────┐
        │ RUNNING │ ◄── Claude Code session live
        └─────────┘
             │
             │
   ┌─────────┼─────────┐
   ▼         ▼         ▼
┌────────┐ ┌──────┐  ┌────────┐
```

```
 │ PAUSED  │  │ERROR │   │STOPPED │
 └─────────┘  └──────┘   └────────┘
      │ POST /resume
      ▼
  [back to RUNNING]
```

18. Database Schema (Key Tables)

Table	Purpose
`agents`	Master agent records (status, manifest, model_type, pending_improvements)
`agent_backtests`	Backtest runs with versioning, metrics, model_selection
`agent_sessions`	Every Claude Code session (parent_agent_id, position_ticker, session_role)
`agent_trades`	Live trades with decision_status (accepted/rejected/paper/watchlist)
`agent_messages`	Inter-agent knowledge sharing
`agent_logs`	Per-agent log entries
`agent_chat_messages`	Dashboard chat history (user ↔ agent)
`system_logs`	System-wide log entries
`notifications`	WhatsApp/dashboard notifications
`watchlist`	Paper trading positions tracked for simulated P&L
`connectors`	Discord/Reddit/Twitter/UW credentials
`connector_agents`	Many-to-many linking agents to connectors
`trading_accounts`	Robinhood broker accounts
`channel_messages`	Ingested Discord messages
`trade_signals`	Parsed trading signals
`positions`	Open trading positions

19. File System Layout

```
data/
├── backtest_{agent_id}/
│   ├── config.json
│   ├── latest.json                # Pointer to latest version
│   └── output/
│       ├── v1/                    # First backtest run
│       ├── v2/                    # Re-runs (non-destructive)
│       │   ├── transformed.parquet
│       │   ├── enriched.parquet
│       │   ├── models/
│       │   ├── patterns.json
│       │   └── live_agent/manifest.json
│       └── v3/
│
├── live_agents/
│   └── {agent_id}/
│       ├── CLAUDE.md              # Rendered live agent instructions
│       ├── config.json
│       ├── tools/                 # Python tool scripts
│       ├── skills/                # Markdown skill files
│       ├── models/                # Copied trained models
│       ├── pending_tasks.json     # User instructions queue
│       ├── positions.json
│       ├── watchlist.json
│       └── positions/             # Sub-agent working dirs
│           └── AAPL_20260406_143000/
│               ├── CLAUDE.md
│               ├── position.json
│               └── tools/
│
└── supervisor/
    └── 20260406/
        ├── CLAUDE.md
        ├── tools/
        └── results.json
```

20. Dashboard (React Frontend)

`apps/dashboard/` — React 18 + Vite + TypeScript + Tailwind CSS + Radix UI + TanStack Query

Key pages: - **Agents List** — all agents with status, win rate, P&L - **Agent Dashboard** — per-agent view with tabs: - **Live** — real-time status, recent trades, equity curve - **Backtesting** — metrics, patterns, model comparison - **Feed** — raw Discord messages from connected channels - **Logs** — agent system logs with SSE streaming - **Chat** — direct conversation with the Claude agent - **Trades** — trade history with decision audit trail - **Portfolio** — positions, P&L, risk metrics - **Connectors** — manage Discord/Reddit/Twitter/Robinhood connections - **Settings** — risk parameters, notification preferences

---

21. API Endpoints Reference

Endpoint	Method	Purpose
`/api/v2/agents`	POST	Create agent + start backtest
`/api/v2/agents/{id}`	GET	Agent details
`/api/v2/agents/{id}/approve`	POST	Approve backtest → spawn live agent
`/api/v2/agents/{id}/pause`	POST	Pause running agent
`/api/v2/agents/{id}/resume`	POST	Resume paused agent
`/api/v2/agents/{id}/stop`	POST	Stop agent permanently
`/api/v2/agents/{id}/chat`	GET/POST	Chat with agent
`/api/v2/agents/{id}/channel-messages`	GET	Discord feed for agent
`/api/v2/agents/{id}/channel-messages/backfill`	POST	On-demand Discord fetch
`/api/v2/agents/{id}/logs`	GET	Agent logs
`/api/v2/agents/{id}/logs/stream`	GET	SSE live log tail
`/api/v2/agents/{id}/trades`	GET	Trade history
`/api/v2/agents/{id}/spawn-position-agent`	POST	Spawn position monitor
`/api/v2/agents/{id}/instruct`	POST	Send instruction to agent
`/api/v2/agents/{id}/runtime-info`	GET	Host, PID, working dir, uptime

Endpoint	Method	Purpose
`/api/v2/agents/{id}/activity-feed`	GET	Unified logs + trades + messages
`/api/v2/agents/graph`	GET	Agent topology for visualization
`/api/v2/agents/morning-briefing`	POST	Trigger morning routine
`/api/v2/agents/supervisor/run`	POST	Trigger AutoResearch
`/api/v2/scheduler/status`	GET	Scheduler + ingestion health
`/api/v2/scheduler/ingestion/refresh`	POST	Restart dead connectors
`/webhook/whatsapp`	POST	Incoming WhatsApp handler

---

22. Security & Sandboxing

Each agent has a `.claude/settings.json` that controls permissions:

ALLOWED: - Run Python scripts in `tools/` - Make HTTP calls to Phoenix API and external services - Read/write within its own working directory

DENIED: - Access files outside its working directory - Run destructive commands (`rm -rf /` , `git push --force`) - Access other agents' data

Credential security: - All credentials encrypted with Fernet (AES-128-CBC) - Decrypted only at runtime in memory - Robinhood sessions persisted via pickle with 24-hour expiry - TOTP auto-generation for 2FA (no manual device approval)

---

23. Deployment Architecture

Deployed via **Coolify** on a VPS:

```
Coolify (Git-based Deploy)
    │ push to main → auto-deploy
    ▼
┌──────────────────────────────────────┐
```

```
│ VPS (69.62.86.166)                  │
│                                     │
│  ┌──────────────────────┐           │
│  │ phoenix-api          │ :8011     │
│  │ (FastAPI + agents)   │           │
│  └──────────┬───────────┘           │
│             │                       │
│  ┌──────────┼──────────┐            │
│  ▼          ▼          ▼            │
│ Postgres  Redis    Dashboard        │
│  :5432    :6379      :3000          │
└─────────────────────────────────────┘
```

- `docker-compose.coolify.yml` defines the service stack
- `apps/api/Dockerfile` builds the API container
- `apps/api/entrypoint.sh` runs migrations on startup, then `uvicorn`

---

24. Connecting to Running Agents

Every Claude Code agent is a subprocess of the `phoenix-api` Docker container.

```
# SSH to VPS
ssh root@69.62.86.166

# Find API container
API=$(docker ps --format '{{.Names}}' | grep phoenix-api | head -1)

# Exec into container
docker exec -it $API bash

# List agent working directories
ls /app/data/live_agents/

# See running Claude processes
ps auxf | grep claude

# Watch an agent's activity
docker logs -f $API 2>&1 | grep "<agent-id>"

# Read agent's state files
cat /app/data/live_agents/<id>/positions.json
cat /app/data/live_agents/<id>/pending_tasks.json
cat /app/data/live_agents/<id>/CLAUDE.md
```

25. Debugging & Troubleshooting

Problem	Solution
Agent says RUNNING but no activity	Check if Claude process is alive: `ps aux | grep claude`
Backtest stuck	Check `latest.json` for version, inspect latest output files
Chat not responding	Check API logs for timeout/SDK errors
Discord messages not in Feed	Call `POST /scheduler/ingestion/refresh` or use backfill endpoint
"Credit balance too low"	This is Anthropic API billing, not Phoenix. Add credits at console.anthropic.com
Agents auto-shutting off	Check heartbeat (30min timeout). Agents auto-restart on stale detection.
Robinhood 2FA prompts	Set `RH_TOTP_SECRET` env var for automatic TOTP generation

26. Technology Stack

Component	Technology
Backend API	Python 3.11+, FastAPI, Uvicorn
Frontend	React 18, Vite, TypeScript, Tailwind CSS, Radix UI
Database	PostgreSQL (SQLAlchemy async ORM, Alembic migrations)
Cache/Pub-Sub	Redis (Streams, Pub/Sub)
AI Agents	Claude Code SDK (`claude-agent-sdk`)
ML Training	XGBoost, LightGBM, CatBoost, PyTorch (LSTM/Transformer/TFT/TCN)
Broker	Robinhood via `robin_stocks` + custom MCP server
Deployment	Docker, Coolify (git-push deploy)

Component	Technology
Testing	pytest, Vitest, Playwright
Linting	Ruff (Python), ESLint (TypeScript)
Notifications	WhatsApp Cloud API, WebSocket

27. Future Roadmap

Phase	Feature
V2	Reddit agent, Twitter agent, Unusual Whales agent with full tool access
V3	Plain-English strategy builder ("buy TQQ/SQQQ based on 8/24 EMA crossover")
V4	Multi-agent teams with shared strategy meetings
V5	AutoResearch (Karpathy-style) — agents that improve themselves daily
V6	Distributed agent execution across multiple VPS nodes
V7	Terminal tab in dashboard — SSH directly into agent workspace

Phoenix is not "code that trades" — it's an army of Claude Code agents that read instructions, run Python tools, share knowledge, and self-improve. The Python services exist to spawn agents, persist state, route messages, serve the dashboard, and dispatch notifications. The actual intelligence lives in the agents.